

RA1 - Actividades Obligatorias

David Cueli Chavarría

Acceso al repositorio de las actividades en GitHub

https://github.com/dcueli-com/applied_bigdata on [develop branch](#)

1 Contexto

En estas actividades trabajarás con herramientas clave del análisis de datos en Big Data: **Pandas, Polars, PySpark y MongoDB**.

El objetivo es que comprendas tanto el tratamiento de datos estructurados como no estructurados, así como las diferencias de rendimiento y uso entre distintas tecnologías.

2 Ejercicio 1: Comparación de Pandas, Polars y PySpark con MovieLens 20M

2.1 Contexto

Trabajarás con el **superdataset de MovieLens 20M**, que contiene:

- userId
- movieId
- rating
- rating timestamp
- title
- genres

2.2 Objetivos

- Comparar el rendimiento de Pandas, Polars y PySpark.
- Analizar operaciones de filtrado, agregación y ordenación.
- Medir tiempos de ejecución.
- Entender el uso de LazyFrames en Polars.

2.3 Entornos

- Pandas en Google Colab.
- Polars en Google Colab (modo normal y LazyFrame).
- PySpark en Databricks Community Edition.

2.4 Tareas a realizar

- Cargar el dataset movielens superdataset.csv.

The screenshot shows a Visual Studio Code interface with a terminal window open. The terminal displays the output of three different Python scripts used to load a dataset from a CSV file.

Pandas Performance

```
> python pandas/compare.py
+ =====
+ 
+ Loaded <ratings.csv> dataset
+ Time to load the dataset with Pandas: 7.3739 seconds
+ 
```

	userId	movieId	rating	timestamp
0	1	2	3.5	1112486027
1	1	29	3.5	1112484676
2	1	32	3.5	1112484819
3	1	47	3.5	1112484727
4	1	50	3.5	1112484580

Polars Performance

```
> python polars/compare.py
+ =====
+ 
+ Loaded <ratings.csv> dataset
+ Time to load the dataset with Polars: 0.2545 seconds
+ 
```

shape: (5, 4)

userId	movieId	rating	timestamp
i64	i64	f64	i64
1	2	3.5	1112486027
1	29	3.5	1112484676
1	32	3.5	1112484819
1	47	3.5	1112484727
1	50	3.5	1112484580

PySpark Performance

```
> python pyspark/compare.py
WARNING: Using incubator modules: jdk.incubator.vector
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
26/04/22 18:54:22 WARN Utils: Your hostname, UNKNOWN-DESKTOP-DEVICE, resolves to a loopback address: 127.0.1.1; using 10.255.255.254 instead (on interface lo)
26/04/22 18:54:22 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another address
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/04/22 18:54:24 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
+ =====
+ 
+ Loaded <ratings.csv> dataset
+ Time to load the dataset with PySpark: 17.3581 seconds
+ 
```

userId	movieId	rating	timestamp
1	2	3.5	1112486027
1	29	3.5	1112484676
1	32	3.5	1112484819
1	47	3.5	1112484727
1	50	3.5	1112484580

only showing top 5 rows

src > activities > lesson_01 > 01 > notebooks > 2_4_analysis_dataset.ipynb > # 2. Filtrar películas de género "Action" con rating ≥ 4

Generate + Code + Markdown | Run All Clear All Outputs | Outline ... Select Kernel

```
# 1. Cargar el dataset movielens superdataset.csv (ratings.csv in PATH_RATINGS)
# =====

df_pd = {}
df_pl = {}
df_sp = {}

print(f"+ =====")
print(f"+ Load '{PATH_RATINGS}' from the movielens dataset ")
print(f"+ -----")
LineBreak(1)
print(f"+ Loading with Pandas...")

tInit = time.time()
df_pd['ratings'] = pd.read_csv(PATH_RATINGS)
tEnd = time.time()
df_pd['movies'] = pd.read_csv(PATH_MOVIES)

print(f"+ Time to load the dataset with Pandas: {tEnd - tInit:.4f} seconds")
SectionBreak()
print(f"+ Loading with Polars...")

tInit = time.time()
df_pl['ratings'] = pl.read_csv(PATH_RATINGS)
tEnd = time.time()
df_pl['movies'] = pl.read_csv(PATH_MOVIES)

print(f"+ Time to load the dataset with Polars: {tEnd - tInit:.4f} seconds")
SectionBreak()
print(f"+ Loading with PySpark...")

tInit = time.time()
df_sp['ratings'] = spark.read.csv(PATH_RATINGS, header=True, inferSchema=True)
countSp = df_sp['ratings'].count()
tEnd = time.time()
df_sp['movies'] = spark.read.csv(PATH_MOVIES, header=True, inferSchema=True)
# Only if we want to force the keep dataset in cache
# df_sp.cache()

print(f"+ Time to load the dataset with PySpark: {tEnd - tInit:.4f} seconds for {countSp} rows")
LineBreak(1)
print(f"+ =====")
```

Python

```
+ =====
+ Load '../dataset/ml-20m/ratings.csv' from the movielens dataset
+ -----
+ Loading with Pandas...
+ Time to load the dataset with Pandas: 7.9170 seconds
+ -----
+ Loading with Polars...
+ Time to load the dataset with Polars: 0.1830 seconds
+ -----
+ Loading with PySpark...

+ Time to load the dataset with PySpark: 2.6293 seconds for 20000263 rows
+
+ =====
```

- Filtrar películas de género "Action" con rating ≥ 4 .

```

File Edit Selection View Go ... 2_4_analysis_dataset.ipynb - applied_bigdata [WSL: Debian] - Visual Studio Code
.../polars compare.py .../pyspark unified.py 2_4_analysis_dataset.ipynb X 2_5_polar_lazyframe.ipynb README.md
src > activities > lesson_01 > 01 > notebooks > 2_4_analysis_dataset.ipynb > # 2. Filtrar películas de género "Action" con rating  $\geq 4$ 
Generate + Code + Markdown Run All Clear All Outputs Outline Select Kernel

# 2. Filtrar películas de género "Action" con rating  $\geq 4$ 
# =====

def GenreFilterAction(pBy: str = "pandas", pGnr: str = "Action", pRat: int = 4):
    print(f"+ -----")
    LineBreak(1)
    print(f"+ Filtering with {pBy.upper()} by {pGnr.upper()} and rating >= {pRat} ...")

    if "pandas" in pBy.lower():
        GenreFilterActionByPandas(pGnr, pRat)
        pass
    if "polars" in pBy.lower():
        GenreFilterActionByPolars(pGnr, pRat)
        pass
    if "pyspark" in pBy.lower():
        GenreFilterActionByPySpark(pGnr, pRat)

def GenreFilterActionByPandas(pGnr: str = "Action", pRat: int = 4):
    tInit = time.time()
    avgRatingsPd = (df_pd['ratings']
                    .groupby('movieId')['rating']
                    .mean()
                    .reset_index()
                    )
    dfJoinedPd = avgRatingsPd.merge(df_pd['movies'], on='movieId')
    bFilterPd = (dfJoinedPd['genres'].str.contains(pGnr)) & (dfJoinedPd['rating'] >= pRat)
    resPd = dfJoinedPd[bFilterPd]
    print(f"+ Pandas result: {len(resPd)} Filtered at Time: {time.time() - tInit:.4f}s")
    LineBreak(1)

def GenreFilterActionByPolars(pGnr: str = "Action", pRat: int = 4):
    tInit = time.time()
    resPl = (df_pl['ratings']
            .group_by("movieId")
            .agg(pl.col("rating").mean().alias("avgRating"))
            .join(df_pl['movies'], on="movieId")
            .filter((pl.col("genres").str.contains(pGnr)) & (pl.col("avgRating") >= pRat))
            )
    print(f"+ Polars result: {resPl.height} Filter Time: {time.time() - tInit:.4f}s")
    LineBreak(1)

def GenreFilterActionByPySpark(pGnr: str = "Action", pRat: int = 4):
    tInit = time.time()
    resSp = (df_sp['ratings']
            .group_by("movieId")
            .agg(F.avg("rating").alias("avgRating"))
            .join(df_sp['movies'], "movieId")
            .filter((F.col("genres").contains(pGnr)) & (F.col("avgRating") >= pRat))
            )
    countSp = resSp.count()
    tEnd = time.time()
    print(f"+ PySpark result: {countSp} Filter Time: {tEnd - tInit:.4f}s")
    LineBreak(1)

print(f"+ =====")
print(f"+ Filtter movies by GENRE with rating  $\geq$  N ...")

GenreFilterAction("pandas", "Action", 4)
GenreFilterAction("polars", "Action", 4)
GenreFilterAction("pyspark", "Action", 4)

print(f"+ =====")

+ =====
+ Filtrar películas de género 'Action' con rating  $\geq 4$ 
+ -----
+
+ Filtering with PANDAS by ACTION and rating >= 4 ...
+ Pandas result: 118 Filtered at Time: 1.1673s
+
+ -----
+
+ Filtering with POLARS by ACTION and rating >= 4 ...
+ Polars result: 118 Filter Time: 0.3525s
+
+ -----
+
+ Filtering with PYSPARK by ACTION and rating >= 4 ...
+ PySpark result: 118 Filter Time: 0.2666s
+
+ =====
  
```

- Calcular media y desviación estándar de ratings por usuario.

```

File Edit Selection View Go ... 2_4_analysis_dataset.ipynb - applied_bigdata [WSL: Debian] - Visual Studio Code
.../polars compare.py .../pyspark unified.py 2_4_analysis_dataset.ipynb 2_5_polar_lazyframe.ipynb README.md
src > activities > lesson_01 > 01 > notebooks > 2_4_analysis_dataset.ipynb > # 2. Filtrar películas de género "Action" con rating ≥ 4
Generate + Code + Markdown | Run All Clear All Outputs | Outline ... Select Kernel

# 3. Calcular media y desviación estándar de ratings por usuario
# =====

def UserStats():
    print(f"+ =====")
    print(f"+ Calculating user stats (ratings average and standard deviation by user)...")
    print(f"+ -----")
    LineBreak(1)

    tInit = time.time()
    userStatsPd = df_pd['ratings'].groupby('userId')['rating'].agg(['mean', 'std'])
    print(f"+ Pandas Stats({len(userStatsPd)}) Time: {time.time() - tInit:.4f}s")

    SectionBreak()

    tInit = time.time()
    userStatsPl = df_pl['ratings'].group_by("userId").agg([
        pl.col("rating").mean().alias("mean"),
        pl.col("rating").std().alias("std")
    ])
    print(f"+ Polars Stats({userStatsPl.height}) Time: {time.time() - tInit:.4f}s")

    SectionBreak()

    tInit = time.time()
    userStatsSp = df_sp['ratings'].groupBy("userId").agg([
        F.avg("rating").alias("mean"),
        F.stddev("rating").alias("std")
    ])
    print(f"+ PySpark Stats({userStatsSp.cache().count()}) Time: {time.time() - tInit:.4f}s")

    LineBreak(1)
    print(f"+ =====")

UserStats()
Python

...
+ =====
+ Calculating user stats (ratings average and standard deviation by user)...
+ -----
+
+ Pandas Stats(138493 ) Time: 0.9745s
+
+ -----
+
+ Polars Stats(138493) Time: 0.2680s
+
+ -----
+
[Stage 98:=====> (51 + 37) / 88]
+ PySpark Stats(138493) Time: 2.8422s
+
+ =====

```

- Ordenar películas por rating promedio.

```
File Edit Selection View Go ... 2_4_analysis_dataset.ipynb - applied_bigdata [WSL: Debian] - Visual Studio Code
.../polars compare.py .../pyspark unified.py 2_4_analysis_dataset.ipynb X 2_5_polar_lazyframe.ipynb README.md
src > activities > lesson_01 > 01 > notebooks > 2_4_analysis_dataset.ipynb > # 2. Filtrar películas de género "Action" con rating ≥ 4
Generate + Code + Markdown | Run All Clear All Outputs | Outline ... Select Kernel

# 4. Ordenar películas por rating promedio
# =====

def SortMoviesByRating():
    print(f"+ =====")
    print(f"+ Sorting movies by average rating...")
    print(f"+ -----")
    LineBreak(1)

    tInit = time.time()
    resSortedPd = df_pd['ratings'].groupby('movieId')['rating'].mean().sort_values(ascending=False)
    print(f"+ Pandas Sort({len(resSortedPd)}) Time: {time.time() - tInit:.4f}s")

    SectionBreak()

    tInit = time.time()
    resSortedPl = (df_pl['ratings']
                  .group_by("movieId")
                  .agg(pl.col("rating").mean())
                  .sort("rating", descending=True)
                  )
    print(f"+ Polars Sort({resSortedPl.height}) Time: {time.time() - tInit:.4f}s")

    SectionBreak()

    tInit = time.time()
    resSortedSp = (df_sp['ratings']
                  .groupBy("movieId")
                  .agg(F.avg("rating").alias("avgRating"))
                  .orderBy(F.desc("avgRating"))
                  )
    countSp = resSortedSp.count()
    tEnd = time.time()
    resSortedSp.show(5)
    print(f"+ PySpark Sort({countSp}) Time: {tEnd - tInit:.4f}s")

    LineBreak(1)
    print(f"+ =====")

SortMoviesByRating()

Python
```

```
...
+ =====
+ Sorting movies by average rating...
+ -----
+
+ Pandas Sort(26744) Time: 1.1072s
+
+ -----
+
+ Polars Sort(26744) Time: 0.3161s
+
+ -----
+
+-----+-----+
|movieId|avgRating|
+-----+-----+
| 100743|      5.0|
| 129293|      5.0|
| 129741|      5.0|
| 103912|      5.0|
| 112790|      5.0|
+-----+-----+
only showing top 5 rows

+ PySpark Sort(26744) Time: 6.2976s
```

- Convertir rating timestamp a fecha.

```

# 5. Convertir rating timestamp a fecha
# =====

def Ts2DateConversion():
    print(f"+ =====")
    print(f"+ Preparing the dataframes (preparation & date conversion)...")
    print(f"+ -----")
    LineBreak(1)

    print(f"+ With Pandas...")
    tInit = time.time()
    df_pd['ratings']['date'] = pd.to_datetime(df_pd['ratings']['timestamp'], unit='s')
    print(f"+ Pandas Date Conversion: {time.time() - tInit:.4f}s")

    SectionBreak()

    print(f"+ With Polars...")
    tInit = time.time()
    df_pl['ratings'] = df_pl['ratings'].with_columns(pl.from_epoch("timestamp", time_unit="s").alias("date"))
    print(f"+ Polars Date Conversion: {time.time() - tInit:.4f}s")

    SectionBreak()

    print(f"+ With PySpark...")
    tInit = time.time()
    df_sp['ratings'] = df_sp['ratings'].withColumn("date", F.from_unixtime("timestamp").cast("date"))
    print(f"+ PySpark Date Conversion: {time.time() - tInit:.4f}s")

    LineBreak(1)
    print(f"+ =====")

Ts2DateConversion()
  
```

```

...
+ =====
+ Preparing the dataframes (preparation & date conversion)...
+ -----
+
+ With Pandas...
+ Pandas Date Conversion: 15.0056s
+
+ -----
+
+ With Polars...
+ Polars Date Conversion: 0.0429s
+
+ -----
+
+ With PySpark...
+ PySpark Date Conversion: 0.0160s
+
+ =====
  
```

- Medir tiempos de ejecución.

En las capturas de pantalla previas, en cada ejercicio, están marcados en cuadros rojos los tiempos de carga sobre un conteo de registros.

2.5 Polars: LazyFrame

- Modo normal: ejecución inmediata.
- LazyFrame: optimiza antes de ejecutar.

```
src > actividades > lesson_01 > 01 > notebooks > 2_5_polar_lazyframe.ipynb > print(f"+ =====")

Generate + Code + Markdown | Run All Restart Clear All Outputs | Jupyter Variables Outline ... .venv (Python 3.13.7)

# 1. Cargar el dataset movielens superdataset.csv (ratings.csv in PATH_RATINGS)
# =====

def PolarsLazyframe():
    tInit = time.time()

    # Creating the lazyframe
    # No read file yet, only inspect the schema
    df_pl_lz['ratings'] = pl.scan_csv(PATH_RATINGS)
    df_pl_lz['movies'] = pl.scan_csv(PATH_MOVIES)

    # No execute yet, only build the query plan
    queryPlan = (df_pl_lz['ratings']
        .group_by("movieId")
        .agg(pl.col("rating").mean().alias("avgRating"))
        .join(df_pl_lz['movies'], on="movieId")
        .filter((pl.col("genres").str.contains("Action")) & (pl.col("avgRating") >= 4))
    )

    # Execute with .collect()
    # Polars optimize the query plan and execute it
    resFinal = queryPlan.collect()

    print(f"+ Polars Lazy Time: {time.time() - tInit:.4f}s")
    print(f"+ Movies found: {resFinal.height}")

[11] ✓ 0.0s Python

# Eager load mode with Polars
# =====

def PolarsEagerframe():
    tInit = time.time()
    # Load & examine immediately
    df_pl_ea['ratings'] = pl.read_csv(PATH_RATINGS)
    df_pl_ea['movies'] = pl.read_csv(PATH_MOVIES)

    # Execute immediately
    resPl = (df_pl_ea['ratings']
        .group_by("movieId")
        .agg(pl.col("rating").mean().alias("avgRating"))
        .join(df_pl_ea['movies'], on="movieId")
        .filter((pl.col("genres").str.contains("Action")) & (pl.col("avgRating") >= 4))
    )

    print(f"+ Polars result: {resPl.height} Filter Time: {time.time() - tInit:.4f}s")

[12] ✓ 0.0s Python

print(f"+ =====")
LineBreak(1)
print(f"+ Analysing Polars Eagerframe...")
print(f"+ -----")
PolarsEagerframe()
SectionBreak()
print(f"+ Analysing Polars Lazyframe...")
print(f"+ -----")
LineBreak(1)
PolarsLazyframe()
LineBreak(1)
print(f"+ =====")

✓ 2.3s Python

+ =====
+
+ Analysing Polars Eagerframe...
+ -----
+ Polars result: 118 Filter Time: 1.8532s
+
+ -----
+
+ Analysing Polars Lazyframe...
+ -----
+
+ Polars Lazy Time: 0.5013s
+ Movies found: 118
+
+ =====
```


2.6 Preguntas

- Diferencias de rendimiento entre herramientas.

Las ejecuciones de las pruebas son todas en local y creo que las diferencias son evidentes.

Pandas sufre considerablemente sobre cualquier operación, sea únicamente al cargar o consultar el dataset sobre el CSV de RATINGS, se penaliza el tiempo de ejecución.

Polars destaca mucho más, es decir, gana en velocidad de ejecución, da igual si es en carga del dataset completo como haciendo consultas sobre él. Creo que aprovecha considerablemente todos los núcleos de la CPU gracias a su implementación en Rust, lo que reduce los tiempos drásticamente.

Por otro lado, **PySpark** presenta una cantidad de tiempo bastante elevado invertido en la preparación de la gestión de la máquina virtual de Java y la capacidad de la máquina local, pero creo que lo suple bastante bien en cargas de trabajo muy pesadas o lo que es lo mismo para datasets realmente gigantescos. En resumen, digo que el tiempo que tarda en obtener la información preguntada es inversamente proporcional al tamaño de datos a consultar.

- Mejora usando LazyFrame.

LazyFrame en Polars supongo que, como es otros ORMs que he usado, es ganancia en eficiencia, pero depende de la consulta a ejecutar. Me explico, a diferencia del modo normal (el Eager), donde las operaciones se ejecutan de inmediato cargando datos en la memoria el modo Lazy, previamente, optimiza el motor para ejecutar la consulta, lo que disminuye considerablemente el tiempo, especialmente en la unión de tablas y filtros de texto. Pero evidentemente estoy explicando cosas que ya sabes.

- Escalabilidad con datasets más grandes.

Supongo que si el tamaño del dataset pudiese superar la capacidad de la memoria física, que es de lo que se trata, el uso, o más bien, el orden de uso de estas herramientas cambiaría:

Yo soy de optimizar siempre, hasta cuando no hace falta, pero queda descartado el usar Pandas para volúmenes de datos gigantes o lo relegaría a la última posición del podio.

Lo que tengo claro que siempre, después de haber estado en clase y haber ejecutado estas prácticas, me quedaría con Polars ya que seguiría siendo eficiente gracias a la capacidad de procesar los recursos de la/s máquina/s. Pero para esto habría que conocer la capacidad del sistema contratado. Si lo anterior se desconoce, pues está claro que aquí, PySpark tiene el podio más alto.

- Uso en la industria.

En el entorno profesional actual, como la conclusión anterior, la elección depende del caso de uso. Pero lo que tengo claro que Pandas, que siendo el estándar para análisis, lo dejaría más bien para un entorno de necesidades rápidas y manipulación de datos pequeños.

Polars, según lo visto en clase, y lo que he leído, está ganando gravedad y atrayendo cada vez más al uso de alto rendimiento.

Pero, por hacer la analogía y terminar ya de escribir (jaja), me gusta el diésel y PySpark no falla si es de muuuy largo recorrido.

3 Ejercicio 2: JSON y MongoDB

3.1 Objetivo

Practicar la creación de documentos JSON y consultas en MongoDB usando **Mongo Playground**.

3.2 Creación de documentos

Crear cinco documentos JSON con:

- cliente id
- nombre
- edad
- ciudad
- compras:
- producto
- precio
- fecha

Restricciones

- Al menos un cliente menor de 30 años.
- Al menos dos clientes menores de 30 o de Madrid.
- Solo tres clientes con compras > 100€.
- Al menos un cliente ha comprado un “Portátil”.
- Un cliente de ciudad distinta de Madrid, Barcelona o Sevilla.

```
[{
  "cliente_id": 1,
  "nombre": "Carlos Rodrigo",
  "edad": 40,
  "ciudad": "Cádiz",
  "compras": [{
    "producto": "Portátil",
    "precio": 1200,
    "fecha": "2024-03-01"
  }, {
    "producto": "Ratón",
    "precio": 25,
    "fecha": "2024-03-05"
  }]
}, {
  "cliente_id": 2,
  "nombre": "Javier Enrique",
  "edad": 42,
  "ciudad": "Madrid",
  "compras": [{
    "producto": "Teclado",
    "precio": 150,
    "fecha": "2024-02-15"
  }]
}, {
  "cliente_id": 3,
  "nombre": "Mario Egea",
  "edad": 29,
  "ciudad": "Madrid",
  "compras": [{
    "producto": "Monitor",
    "precio": 200,
    "fecha": "2024-01-20"
  }]
}]
```

```

}, {
  "cliente_id": 4,
  "nombre": "Jorge Garrido",
  "edad": 43,
  "ciudad": "Barcelona",
  "compras": [{
    "producto": "Funda",
    "precio": 30,
    "fecha": "2024-03-10"
  }]
}, {
  "cliente_id": 5,
  "nombre": "Clara Sánchez",
  "edad": 41,
  "ciudad": "Sevilla",
  "compras": [{
    "producto": "Cable HDMI",
    "precio": 15,
    "fecha": "2024-03-12"
  }]
}]

```

3.3 Inserción

Introducir los documentos en Mongo Playground y verificar su formato.

The screenshot shows the Mongo Playground interface. The 'Database' tab is selected, showing a collection named 'bson'. The 'Query' tab is active, displaying a query that filters documents where the price of a purchase is greater than 100. The 'Result' tab shows the output of the query, which includes three documents. Each document contains an ObjectId, the client's name and city, and a list of purchases with their product, price, and date.

```

1: [{
2:   "cliente_id": 1,
3:   "nombre": "Carlos Rodrigo",
4:   "edad": 40,
5:   "ciudad": "Cádiz",
6:   "compras": [{
7:     "producto": "Portátil",
8:     "precio": 1200,
9:     "fecha": "2024-03-01"
10:  }],
11:   "producto": "Ratón",
12:   "precio": 25,
13:   "fecha": "2024-03-05"
14: }]
15: }, {
16:   "cliente_id": 2,
17:   "nombre": "Javier Enrique",
18:   "edad": 42,
19:   "ciudad": "Madrid",
20:   "compras": [{
21:     "producto": "Teclado",
22:     "precio": 150,
23:     "fecha": "2024-02-15"
24:  }]
25: }, {
26:   "cliente_id": 3,
27:   "nombre": "Mario Egea",
28:   "edad": 29,
29:   "ciudad": "Madrid",
30:   "compras": [{
31:     "producto": "Monitor",
32:     "precio": 200,
33:     "fecha": "2024-01-20"
34:  }]
35: }, {
36:   "cliente_id": 4,
37:   "nombre": "Jorge Garrido",
38:   "edad": 43,
39:   "ciudad": "Barcelona",
40:   "compras": [{
41:     "producto": "Funda",
42:     "precio": 30,
43:     "fecha": "2024-03-10"
44:  }]
45: }, {
46:   "cliente_id": 5,
47:   "nombre": "Clara Sánchez",
48:   "edad": 41,
49:   "ciudad": "Sevilla",
50:   "compras": [{
51:     "producto": "Cable HDMI",
52:     "precio": 15,
53:     "fecha": "2024-03-12"
54:  }]
55: }]

```

```

1: db.collection.find({
2:   "compras.precio": {
3:     $gt: 100
4:   }
5: })

```

```

{
  "_id": ObjectId("5a934e000102030405000000"),
  "ciudad": "Cádiz",
  "cliente_id": 1,
  "compras": [
    {
      "fecha": "2024-03-01",
      "precio": 1200,
      "producto": "Portátil"
    },
    {
      "fecha": "2024-03-05",
      "precio": 25,
      "producto": "Ratón"
    }
  ],
  "edad": 40,
  "nombre": "Carlos Rodrigo"
},
{
  "_id": ObjectId("5a934e000102030405000001"),
  "ciudad": "Madrid",
  "cliente_id": 2,
  "compras": [
    {
      "fecha": "2024-02-15",
      "precio": 150,
      "producto": "Teclado"
    }
  ],
  "edad": 42,
  "nombre": "Javier Enrique"
},
{
  "_id": ObjectId("5a934e000102030405000002"),
  "ciudad": "Madrid",
  "cliente_id": 3,
  "compras": [
    {
      "fecha": "2024-01-20",
      "precio": 200,
      "producto": "Monitor"
    }
  ],
  "edad": 29,
  "nombre": "Mario Egea"
}

```

MongoDB version 8.0.21 - [Report an issue](#) - [About this playground](#)

3.4 Consultas

- Clientes con compras > 100€.
- Nombre y ciudad de compradores de “Portátil”.
- Clientes de Madrid con edad > 30.
- Compras del cliente más joven.
- Clientes con al menos dos compras.

3.5 Explicación

Cada consulta debe incluir una explicación de 3 a 5 líneas.

```
/**
 * =====
 * 1. Clientes con compras > 100€
 * -----
 * Accede al campo 'precio' dentro del Array de 'compras' de cada elemento. El operador '$gt' (greater than)
 * filtra los elementos donde al menos uno cumple la condición
 */

db.collection.find({ "compras.precio": { $gt: 100 } })

/**
 * =====
 *
 *
 * 2. Nombre y ciudad de compradores de 'Portátil'
 * -----
 * Filtro directo sobre 'producto' en el Array del 'compras' de cada elemento y para la salida se indica con
 * un 1 los campos que queremos visualizar y con un 0 el campo _id para excluirlo
 */

db.collection.find({ "compras.producto": "Portátil" }, { "nombre": 1, "ciudad": 1, "_id": 0 });

/**
 * =====
 *
 *
 * 3. Clientes de Madrid con edad > 30
 * -----
 * Operador lógico '$and' para combinar dos criterios sobre los campos de cada elemento del Array raíz
 */

db.clientes.find({ $and: [ { "ciudad": "Madrid" }, { "edad": { $gt: 30 } } ] });

/**
 * =====
 *
 *
 * 4. Compras del cliente más joven
 * -----
 * Se ordena el Array de elementos por el campo o atributo 'edad' de forma ascendente (1) mediante 'sort' y
 * se aplica un 'limit' 1 para quedarnos únicamente con el primer registro de la lista ordenada
 */

db.clientes.find({}, { "compras": 1, "_id": 0 }).sort({ "edad": 1 }).limit(1);

/**
 * =====
 *
 *
 * 5. Clientes con al menos dos compras
 * -----
 * El operador '$where' permite aplicar una condición para evaluar la propiedad '.length' del atributo o
 * campo compras como si de un objeto se tratase, por lo que según se pide en el ejercicio, se filtran los
 * elementos cuyo Array de 'compras' tiene una longitud mayor o igual a 2
 */

db.clientes.find({ $where: "this.compras.length >= 2" });
```

4 Conclusión

Estas actividades permiten trabajar tanto con datos estructurados (Pandas, Polars, PySpark) como no estructurados (MongoDB), desarrollando una visión completa del ecosistema Big Data.